感应加热 HMI 调试与 STM32 接口手册

适用工程　加热 1_调试版_V6.HMI

本手册用于在 USART HMI 模拟器中输入数据检查界面，并将同一套变量和指令接入 STM32。屏幕负责显示、参数输入和按钮请求；测量值、实际温段、运行时间及故障状态由主控提供。

1 先完成一次总览刷新

打开 V6 工程并编译，进入调试，选择“指令发送到 当前模拟器”和 String 模式。先单独执行 page 0，再执行其余代码。文本输入区无需手工追加 FF FF FF；STM32 串口发送必须追加三个二进制结束字节。

```
page 0
valid=1
temp10=1196
target10=1200
bus10=3080
amp10=182
freq100=2010
powerW=860
run=1
stage=2
phaseState=2
locked=1
totalSec=123
remain=456
dirty=1
click bInput,0
```

预期结果：实际温度 119.6℃，目标温度 120.0℃，电功率 860W，频率 20.10kHz，当前温段 2 / 3，已运行 123s，预计剩余 456s。界面不会自行递增时间或切换温段。

输入提交的约定

dirty=1 刷新读数和状态，push=1 保存一组曲线快照，alarmPush=1 提交一次报警变化。每组数据最后发送 click bInput,0，立即执行当前页的输入处理事件。每页还有 200ms 处理定时器作为补充；连续多组数据仍应逐组立即提交。

初始测量值为 --，采样曲线和报警表为空。文中的数据只在你发送指令后出现。

2 读数和运行状态变量

变量名区分大小写，使用十进制整数赋值。带倍率的数值应先在 STM32 中换算为整数；不要直接发送 temp10=119.6。

变量	含义	发送示例及显示
valid	测量值有效标志	1 显示数值；0 显示 --
temp10	实际温度 ×10	1196 → 119.6℃
target10	当前目标温度 ×10	1200 → 120.0℃
mean10	区域平均温度 ×10	1196 → 119.6℃
high10	区域最高温度 ×10	1248 → 124.8℃
low10	区域最低温度 ×10	1142 → 114.2℃
bus10	母线电压 ×10	3080 → 308.0V
amp10	谐振电流 ×10	182 → 18.2A
freq100	工作频率 kHz ×100	2010 → 20.10kHz
powerW	电功率 单位 W	860 → 860W
angle10	调功移相角 ×10	620 → 62.0
phase10	锁相相位差 ×10	-15 → -1.5°
run	运行反馈	0 待机；1 运行
stage	实际温段编号	0 未开始；1～nseg 为温段
phaseState	运行阶段	0 待机；1 升温；2 保温；3 已完成
locked	锁相反馈	0 未锁定；1 已锁定
totalSec	累计运行秒数	123 → 123 s
remain	预计剩余秒数	456 → 456 s

更新完一组变量后，发送 dirty=1 和 click bInput,0。目标反馈 target10 与设定参数 tg1～tg12 是不同变量：前者是当前反馈，后者用于编辑加热方案。

直接修改 tActual.txt="100.5" 只影响当前页文本，换页或刷新后会被变量值重绘。跨页保留的数据应写入上述全局变量。

3 温段参数和按钮请求

分段页每页显示 6 行，最多 12 段。点击编号选中，“插入温段”在选中段之后插入，“删除选中”移动后续参数。运行反馈 run=1 时，参数编辑锁定。

变量	含义	输入范围或约定
nseg	温段数量	1～12；指令修改时配套设置 sel 和 stagepage
tg1～tg12	各段目标温度	整数 ℃，1～999
hd1～hd12	各段保温时间	整数 s，1～9999
plim	功率上限	整数 W，1～9999
mode	控制算法	0 LADRC；1 PID；2 模糊 LADRC
flo / fhi	搜频下限和上限	整数 kHz，1～99，flo < fhi
ov / uv / oc	过压 欠压 过流阈值	整数 V / V / A，1～9999，uv < ov
ot1 / ot2 / ot3	功率管 整流桥 工件温度阈值	整数 ℃，1～999

例：停止状态下配置两段。先发送这些赋值，再单独发送 page 1 重画表格和程序预览。page 指令不要放在同一控件事件代码的中间。

```
run=0
nseg=2
sel=1
stagepage=0
tg1=80
hd1=60
tg2=120
hd2=180
mode=2
plim=1500
page 1
```

cmd	按钮请求	cmd	按钮请求
1	启动加热	2	停止加热
3	应用参数	4	保存参数或方案
5	重新搜频	6	故障复位
7	确认报警	0	无待处理请求

按钮只写 cmd，不直接生成 run、温段或温度反馈。STM32 可用 get cmd 查询，处理后写 cmd=0，并更新反馈变量。cmd 是一个槽位，不是队列；连续快速点击可能覆盖。正式固件应补充请求序号或队列及确认机制，避免旧请求处理完后清除新请求。

4 曲线采样指令

曲线仍使用 line 和 fill 绘图，没有曲线控件。每次 push 保存时间戳以及目标温度、平均温度、最高温度、功率、频率、相位差六个通道。下面两点足以验证折线，执行前先单独进入 page 2。

```
page 2
clearPlot=1
click bInput,0
tfrom=0
tto=60
sampleSec=0
target10=600
mean10=550
high10=580
powerW=900
freq100=2000
phase10=20
push=1
click bInput,0
sampleSec=60
target10=1200
mean10=1160
high10=1210
powerW=860
freq100=2020
phase10=5
push=1
click bInput,0
```

预期温度页出现三条从左向右连接的线段。切换功率、频率或相位差页，会使用同一批快照绘制各自通道。low10、temp10、bus10 等读数不会自动由这些通道推算，需要单独赋值。

时间戳和缓存

sampleSec 使用 0～86400 的整数秒，必须严格递增。相同或倒退的时间戳被拒绝，dataErr=1；可用 get dataErr 查看。接受一组快照后 valid 自动置 1。修改数据但不设置 push，不会进入曲线缓存。

当前保存最近 32 组快照，满后丢弃最早一组。1 秒采样一次时，32 点只跨越约 31 秒；将时间范围设为 600 秒并不会增加历史容量。长时间记录应放在 STM32 缓存中，再按显示需求补充屏幕缓存设计。

5 时间范围和页面操作

页 ID	页面	起点变量	终点变量
2	温度曲线	tfrom	tto
3	功率曲线	pfrom	pto
4	频率曲线	ffrom	fto
6	相位差曲线	afrom	ato

其他页面编号：0 运行总览，1 分段温控，5 保护报警，7 数字键盘。

起点与终点均按秒输入，范围为 0～86400，且起点必须小于终点。修改后会裁剪并重新绘制缓存中的曲线；未缓存的历史数据不会凭空补齐。

```
page 2
tfrom=10
tto=50
dirty=1
click bInput,0
```

指令输入非法范围时显示错误提示，不执行除零绘图。通过键盘修改时，确认按钮会拒绝起点不小于终点的输入。

冻结 恢复 与清空

```
frozen=1
frozen=0
dirty=1
click bInput,0
```

上面第一行是冻结，后面三行是恢复刷新，两种操作应分别执行。冻结只停止曲线重画，输入数据仍进入缓存；换页会按当前缓存重新初始化页面。

```
clearPlot=1
click bInput,0
```

清空后采样计数和最后时间戳归零，可重新从 sampleSec=0 开始；已有测量读数不会自动清空。需要清空读数时另设 valid=0、dirty=1 并提交。

固定纵轴范围：温度 0～150℃；功率 0～1500W；频率 15～25kHz；相位差 -30～30°。超范围的值绘制到边缘。当前版本没有自动纵轴缩放；plim 或搜频边界变化也不会改动这些刻度。

6 报警输入和历史记录

初始报警表为空。先单独执行 page 5；下面第一组触发母线过压，第二组表示同一故障已解除，两组分开执行。

```
page 5
alarmIn=3
alarmOn=1
alarmSec=12
alarmPush=1
click bInput,0


alarmIn=3
alarmOn=0
alarmPush=1
click bInput,0
```

触发后顶部显示母线过压，历史状态为未确认。点击确认报警只改为已确认；解除后顶部恢复为无，历史变为已恢复。故障复位按钮只产生 cmd=6，必须由主控确认故障解除后再发送恢复事件。

alarmIn	报警名称
1	测温数据中断
2	锁相失败
3	母线过压
4	母线欠压
5	谐振过流
6	功率管过温
7	整流桥过温
8	工件超温

alarmSec 为发生时的运行秒数。最近两条记录跨页保留、掉电清空。当前实现只显示一个当前报警码，新活动码替换顶部显示；不是多故障并发报警集合。主控需要多故障同时显示时，应扩展为报警位图或队列。

保护阈值不会在屏幕内自动执行保护判断；它们是可编辑参数。故障检测、停止加热和是否允许复位，应由 STM32 控制程序负责。

7 STM32 串口发送示例

以下示例使用传统 STM32Cube HAL1，假定 CubeMX 已生成 main.h 和 huart1。把句柄改成实际串口，先按工程初值 9600 波特率设置，两端数据位、校验位和停止位保持一致。每条命令是 ASCII 文本，末尾紧跟三个 0xFF 字节，不发送字符串结束符或额外换行。

```
#include "main.h"
#include <stdint.h>
#include <string.h>
#include <stdio.h>
#include <inttypes.h>
extern UART_HandleTypeDef huart1;

HAL_StatusTypeDef Hmi_Send(const char *cmd)
{
    uint8_t frame[256];
    size_t n = strlen(cmd);
    if (n > sizeof(frame) - 3U) return HAL_ERROR;
    memcpy(frame, cmd, n);
    frame[n] = frame[n + 1] = frame[n + 2] = 0xFF;
    return HAL_UART_Transmit(&huart1, frame,
                             (uint16_t)(n + 3U), 300U);
}

HAL_StatusTypeDef Hmi_SetI32(const char *name, int32_t v)
{
    char cmd[96];
    int n = snprintf(cmd, sizeof(cmd),
                     "%s=%" PRId32, name, v);
    if (n < 0 || (size_t)n >= sizeof(cmd))
        return HAL_ERROR;
    return Hmi_Send(cmd);
}

HAL_StatusTypeDef Hmi_UpdateTemperature(int32_t temp10)
{
    if (Hmi_SetI32("temp10", temp10) != HAL_OK)
        return HAL_ERROR;
    if (Hmi_SetI32("valid", 1) != HAL_OK)
        return HAL_ERROR;
    if (Hmi_SetI32("dirty", 1) != HAL_OK)
        return HAL_ERROR;
    return Hmi_Send("click bInput,0");
}
```

例如调用 Hmi_UpdateTemperature(1196)，显示 119.6℃。发送目标温度、功率等也采用 Hmi_SetI32。添加曲线点时先发时间戳和所有通道，再发 push=1，最后提交。每一步应检查返回值，部分发送失败后不要继续提交不完整快照。

这段代码使用阻塞发送，放在通信任务或主循环中，不要直接放进高频加热控制中断。9600 波特率下应按实际数据量和屏幕执行速度节流；可改为带队列的 DMA 发送，但发送缓冲区必须保留到传输完成。

8 STM32 读取请求和参数

发送 Hmi_Send("get cmd") 查询按钮请求；数值返回帧固定为 8 字节，格式如下。读取其他数值参数如 plim、tg1、hd1 也使用 get，返回中没有变量名称，因此同一时间只保留一个未完成查询。

字节位置	含义
0	0x71 数值返回标识
1～4	32 位数值 小端排列 低字节在前
5～7	0xFF 0xFF 0xFF 结束符

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <limits.h>

bool Hmi_DecodeNumber(const uint8_t *f, size_t n,
                      int32_t *value)
{
    if (!f || !value || n != 8U || f[0] != 0x71)
        return false;
    if (f[5] != 0xFF || f[6] != 0xFF || f[7] != 0xFF)
        return false;
    uint32_t u = (uint32_t)f[1]
        | ((uint32_t)f[2] << 8)
        | ((uint32_t)f[3] << 16)
        | ((uint32_t)f[4] << 24);
    *value = (u <= INT32_MAX) ? (int32_t)u
        : -1 - (int32_t)(UINT32_MAX - u);
    return true;
}
```

例如 71 01 00 00 00 FF FF FF 表示数值 1。该函数只解码已组好的完整帧，不负责串口逐字节收包。DMA 或中断接收端需要缓存、识别帧类型、处理超时，并将完整帧交给它。数值内容本身可能含 FF FF FF，所以不能只遇到三个 FF 就截断所有帧。

建议处理顺序

查询 cmd → 收到有效数值 → 按请求处理并读取所需参数 → 更新 run、stage 等反馈 → 提交 dirty → 清理已处理请求。处理“应用参数”时读取 nseg、mode、plim、tg1～tg12、hd1～hd12；保护参数同样逐项读取后复核范围。

调试时可发送 bkcmd=3 查看指令成功或失败返回。它会增加应答帧，接收程序要区分这些短应答和 0x71 数值帧。若工程选择 bkcmd=0 降低应答量，get 的数值响应仍需接收处理。当前接口没有请求序号，避免同时排队多个 get 或假设 cmd 不会被后一次点击覆盖。

9 联调检查和接口边界

检查项	操作和预期
基础读数	发送第 1 页代码，核对温度倍率、频率小数和运行状态。
负数相位	phase10=-15，valid=1，dirty=1，提交后显示 -1.5°。
温段分页	停止后插入到 12 段，第 1 页显示 1～6，第 2 页显示 7～12。
范围校验	起止秒数相等或倒置应被拒绝；有效范围只重画缓存内数据。
采样缓存	按递增时间发送至少两点，测试冻结、恢复、清空和第 33 点替换最早点。
报警状态	触发 → 确认 → 主控发送恢复，确认按钮不能让活动故障自行消失。
按钮请求	点击按钮后 get cmd，应得到第 3 页列出的请求号。
掉电行为	本版本变量、方案和曲线历史均为内存数据，重启恢复初值。

常见问题

数值不变：检查大小写、valid、dirty 及 click bInput,0 是否提交。曲线不出现：检查是否至少两点、push 是否置位、时间戳是否递增、范围是否包含这些点，以及 frozen 是否为 0。

参数已改但界面没变：全局参数输入后重新进入对应设置页，例如 page 1。参数输入框和程序预览主要在页面初始化时更新；只写 dirty 不保证重画设置表格。

STM32 能发但屏幕不响应：先核对两端串口配置，再检查每条文本命令末尾是否是三个二进制 FF。不要把字符串 "FF FF FF" 当结束字节，也不要只在整个多条命令块末尾追加一次。

本版本边界

本工程是界面和调试接口，不含加热控制算法、硬件保护执行、掉电保存和完整业务协议。采样缓存为 32 组，报警历史为两条，当前活动报警为一个码；接入正式固件时应按实际需求扩展。

V6 已完成离线结构、事件和布局检查；尚未完成官方编辑器编译及实屏验收。上电联调前，以官方编译结果和实际串口回包确认行为。

指令依据和实现位置

屏幕指令格式依据当前软件附带 help/help.pdf 中的“串口 HMI 指令集”、click、get 和设备返回数据格式。工程变量与事件以界面制作目录的 Program.s、debug_runtime.py、界面定义.json 为准。

STM32 HAL1 函数签名参考 ST 官方 HAL UART I/O operations migration。采用 HAL2 的芯片系列需按对应驱动修改句柄类型和函数签名。

在线参考 ST HAL UART I/O operations migration